David Matei

Internship Dynamics & Azure / Full Stack

2026

Solutions

1. 1 The UML diagram used by the coffee shop application

Coffee Shop

Represents a physical shop location in the chain.

- Stores identification and address details.
- Acts as the context in which baristas work and orders are created.

Barista

Represents an employee who prepares drinks.

- Used to track who prepared an order.

Customer <abstract>

Represents a customer enrolled in the loyalty program.

- Holds the points balance.
- Defines the common interface for earning points and redeeming points.
- Made abstract because the points earning rule depends on membership tier.
  O**perations:**
- earnPoints(amountEUR) (this is where the different classes calculate points differently)
- redeemPoints(ingests the calculated points into the private variables inside the class)

RegularCustomer / GoldCustomer

Membership tiers in the loyalty program.

- RegularCustomer earns **1 point per euro**.
- GoldCustomer earns **2 points per euro**.

I choose this design because it makes the most sense when trying to compute the points and redeeming them. This supports a clean OOP separation of responsibilities.

Order

- Stores timestamp and total price.
- Stores points earned and points redeemed for auditability.
- Contains one or more order items.
- calculateTotal()
- calculatePointsEarned()
- Belongs to one Customer, belongs to one CoffeeShop, prepared by one Barista, contains one or more OrderItem

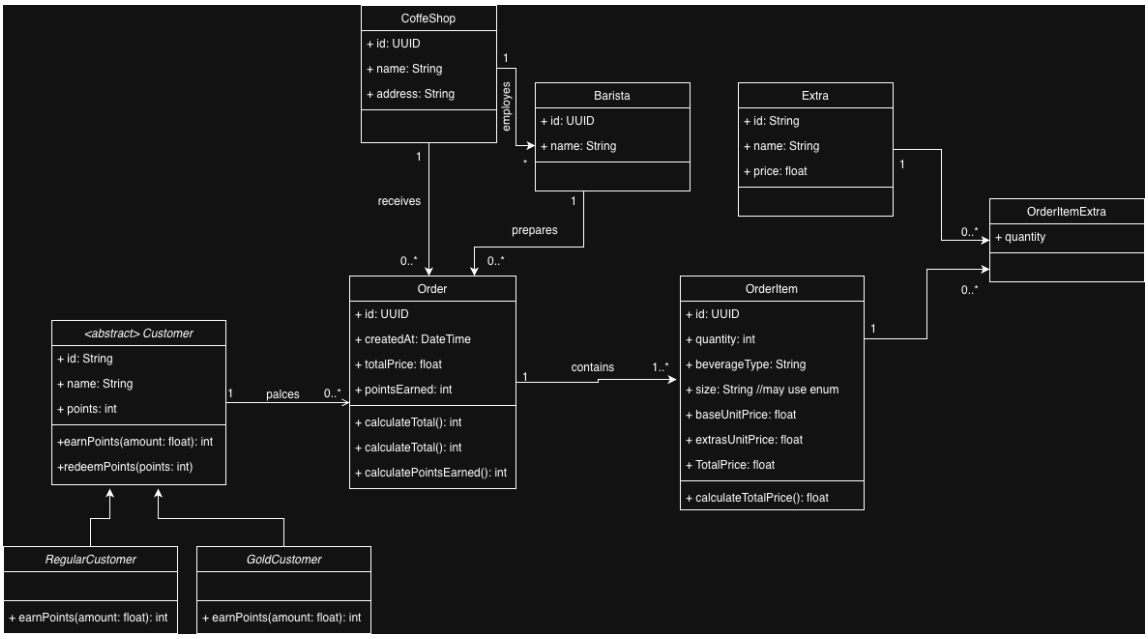
OrderItem

Represents one line item in an order (a specific drink configuration).

- Identifies beverage by beverageType and size.
- Supports quantity
- Tracks base unit price and extras cost.
- Can have multiple extras.

I avoided subclassing coffee types to keep the model flexible. Since coffee types differ only by attributes such as size and price, introducing inheritance would add unnecessary complexity. This approach allows the system to adapt easily if the product range changes, without requiring structural redesign.

Based on this design, I created the UML diagram for the system.

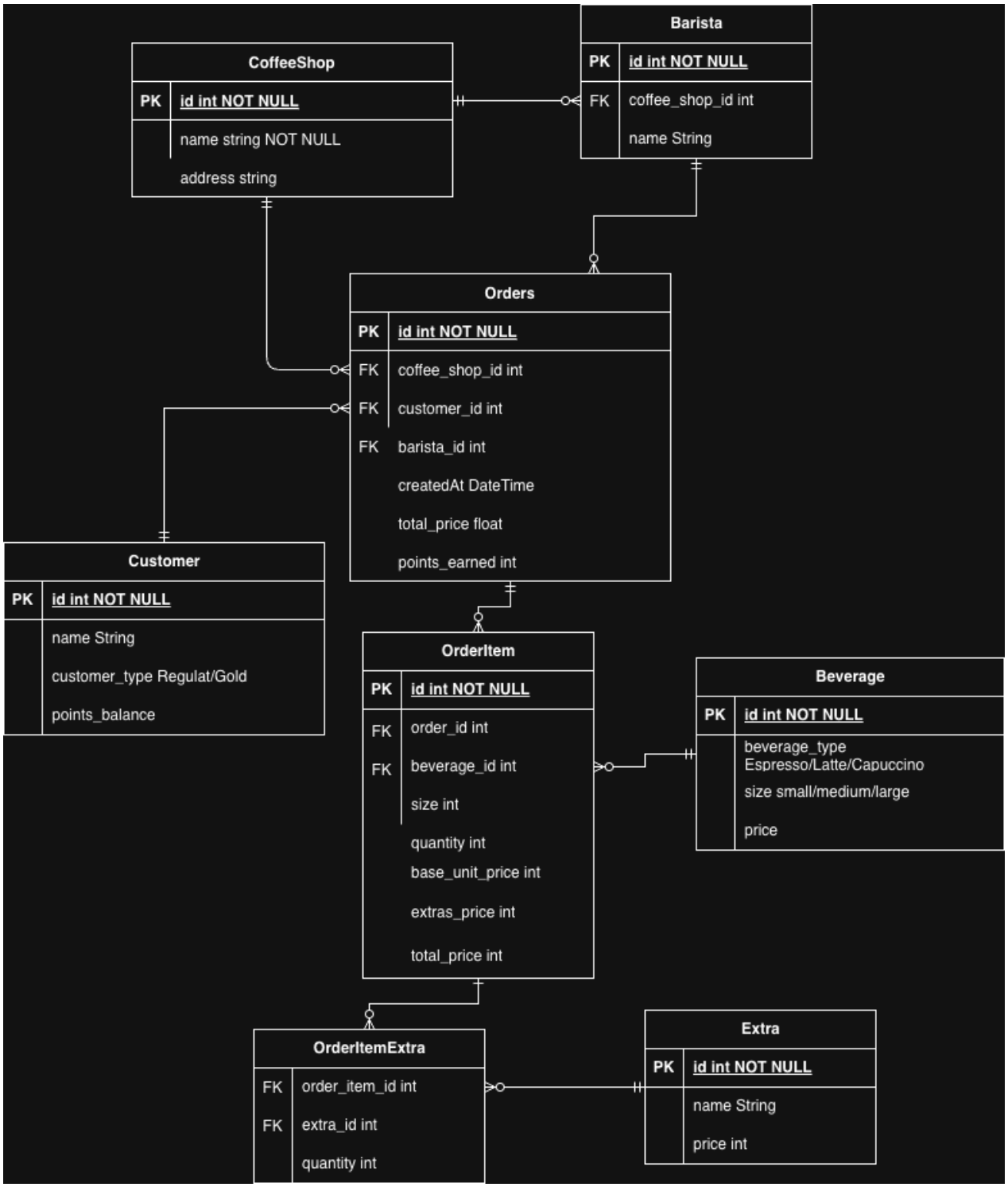


1.2 ER diagram for Sarah's coffee shop:

This ER diagram was created based on the story and the UML model to reflect how the system would be structured in a relational database.

Orders are the central element, connecting customers, baristas, and coffee shops. Each order contains one or more items representing beverages defined by type and size, with optional extras modelled through a junction table.

Customer loyalty is represented through Membership type and points balance, while points earned are tracked at the order level.


CoffeeShop
PK id int NOT NULL
name string NOT NULL
address string
Barista
PK id int NOT NULL
FK coffee_shop_id int
name String
Orders
PK id int NOT NULL
FK coffee_shop_id int
FK customer_id int
FK barista_id int
createdAt DateTime
total_price float
points_earned int
Customer
PK id int NOT NULL
name String
customer_type Regulat/Gold
points_balance
OrderItem
PK id int NOT NULL
FK order_id int
FK beverage_id int
size int
quantity int
base_unit_price int
extras_price int
total_price int
Beverage
PK id int NOT NULL
beverage_type Espresso/Latte/Capuccino
size small/medium/large
price
OrderItemExtra
FK order_item_id int
FK extra_id int
quantity int
Extra
PK id int NOT NULL
name String
price int

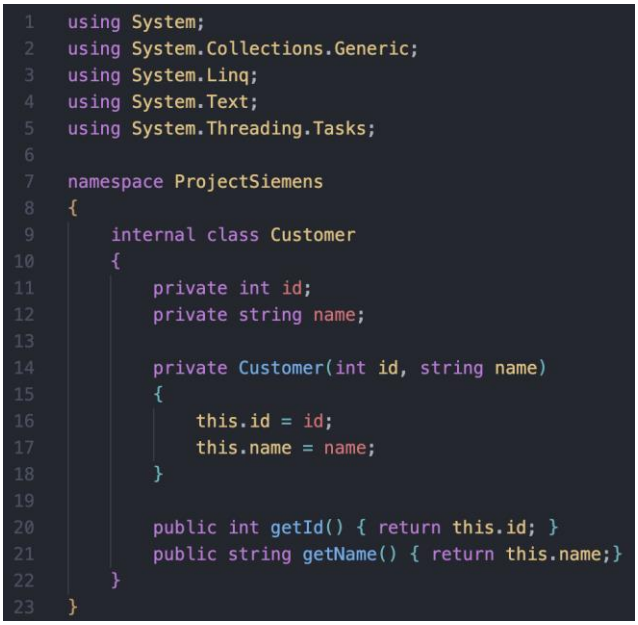
2.1 Class models

I created the core entities required by the system using a simple OOP design with private variables and getters/setters to preserve encapsulation and maintain clear data ownership.

An Order belongs to a Customer and contains a list of OrderItems. Monetary values are represented using decimal.

The class models for the objects:

- Customer.cs



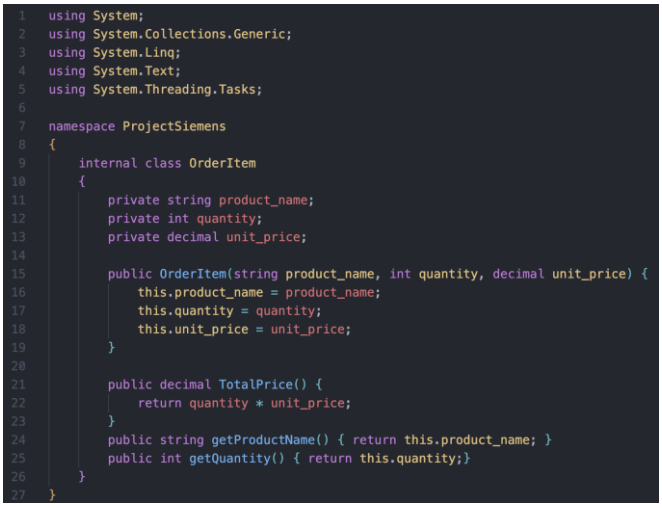
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ProjectSiemens
{
    internal class Customer
    {
        private int id;
        private string name;

        private Customer(int id, string name)
        {
            this.id = id;
            this.name = name;
        }

        public int getId() { return this.id; }
        public string getName() { return this.name;}
    }
}
```

- OrderItem.cs



```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ProjectSiemens
{
    internal class OrderItem
    {
        private string product_name;
        private int quantity;
        private decimal unit_price;

        public OrderItem(string product_name, int quantity, decimal unit_price) {
            this.product_name = product_name;
            this.quantity = quantity;
            this.unit_price = unit_price;
        }

        public decimal TotalPrice() {
            return quantity * unit_price;
        }
        public string getProductName() { return this.product_name; }
        public int getQuantity() { return this.quantity;}
    }
}
```

- Order.cs

```
namespace ProjectSiemens
{
    internal class Order
    {
        private int id;
        private Customer Customer;
        private List<OrderItem> Items;
        private bool discounted;

        public Order(int id, Customer customer) {
            this.id = id;
            this.Customer = customer;
            this.discounted = false;

            Items = new List<OrderItem>();
        }

        public decimal CulculateTotal()
        {
            // we use this function to calculate the subtotal of the order without the discount
            decimal price = 0;
            foreach (OrderItem item in Items)
            {
                price += item.TotalPrice();
            }
            return price;
        }

        public decimal CalculateFinalPrice()
        {
            // this is the final price with the discount applied
            decimal price = this.CulculateTotal();
            if (price > 500)
            {
                price = price - price * 0.9m;
                this.discounted = true;
            }
            return price;
        }

        public Customer getCustomer() { return this.Customer; }
        public bool getDiscounted() {

            if (!discounted)
            {
                this.CalculateFinalPrice(); //we make sure that the discounted flag is set if discounted is false
            }
            return discounted;
        }
        public List<OrderItem> getItems()
        {
            return Items;
        }
    }
}
```

2.2 Final price calculation

The total is calculated inside the Order by summing all items (quantity × unit price). If the total exceeds 500€, a 10% discount is applied to the entire order.

- In Order.cs

```
        public decimal CulculateTotal()
        {
            // we use this function to calculate the subtotal of the order without
            decimal price = 0;
            foreach (OrderItem item in Items)
            {
                price += item.TotalPrice();
            }
            return price;
        }

        public decimal CalculateFinalPrice()
        {
            // this is the final price with the discount applied
            decimal price = this.CulculateTotal();
            if (price > 500)
            {
                price = price - price * 0.9m;
                this.discounted = true;
            }
            return price;
        }
```

2.3 Top spending customer

I used a Manager that holds a list of orders. The method aggregates the final price of each order per customer and returns the name of the customer with the highest total spend.

- Manager.cs

```
public string GetTopSpendingCustomer()
{
    // i made 2 dictionaries to stored firstly the customers and then go through them and find they most
    Dictionary<int, decimal> totals = new Dictionary<int, decimal>();
    Dictionary<int, string> names = new Dictionary<int, string>();

    foreach(var order in orders)
    {
        int customerId = order.getCustomer().getId();
        decimal finalPrice = order.CalculateFinalPrice();

        if (!totals.ContainsKey(customerId))
        {
            totals[customerId] = 0;
            names[customerId] = order.getCustomer().getName();
        }

        totals[customerId] = finalPrice;
    }

    decimal max = 0;
    int bestCustomer = 1;

    foreach(var entry in totals) {
        if(entry.Value > max)
        {
            max = entry.Value;
            bestCustomer = entry.Key;
        }
    }

    if(bestCustomer == -1)
    {
        return "";
    }
    return names[bestCustomer];
}
```

2.4 Popular products + total quantity sold

The method iterates through all orders and their items, aggregating the total quantity sold per product and returning a mapping of ProductName to TotalQuantitySold.

Code:

- Manager.cs

```
60          public Dictionary<string, decimal> GetPopularProducts()
61          {
62              // i make a disctionary to send store the products and they're quantity from all of the orders
63              Dictionary<string, decimal> products = new Dictionary<string, decimal>();
64
65              foreach(var order in orders) {
66                  foreach(var item in order.getItems())
67                  {
68                      if (!products.ContainsKey(item.getProductName())){
69                          products[item.getProductName()] = 0;
70                      }
71
72                      products[item.getProductName()] += item.getQuantity();
73                  }
74              }
75              return products;
76          }
77      }
78  }
```

This design keeps the domain models simple while centralizing analysis logic in the manager layer, making the solution clear, maintainable, and easy to extend.